

第十九届“挑战杯”全国大学生课外学术科技 作品竞赛“揭榜挂帅”专项赛

项目设计报告

选题名称

SH-10 无人机双机协同自主搜救任务

作品名称

面向自主搜救任务的双机系统智能决策算法研究

学校

北京航空航天大学

成员

郑宇杰、传真奇、张家文、杨佳宇轩

2025 年 8 月 15 日

目录

1	项目简介	3
2	项目背景	3
3	项目目标	3
4	项目实施	4
4.1	阶段一：任务启动与广域巡航	4
4.2	阶段二：目标搜索、识别与信息共享	4
4.3	阶段三：精准抵近与末端作业	5
4.4	阶段四：任务完成与返航	5
5	项目结构	5
5.1	项目文件结构	5
5.2	部分介绍	6
6	开发过程	6
6.1	整体流程	6
6.2	项目分工	7
7	文件说明	7
7.1	vtol_commander.py（阶段一）	7
7.1.1	整体路线预览	8
7.1.2	起飞降落逻辑	8
7.1.3	避障策略	9
7.2	detect_stage1.py（阶段二）	9
7.3	iris_commander.py（阶段三）	10
7.3.1	整体路线预览（示例）	11
7.3.2	控制逻辑	11
7.3.3	避障策略	11
7.4	detect_white.py & detect_red.py（阶段四）	11
8	关键代码及注释	12
9	创新点	12
9.1	异构双机协同任务规划与 GNC 一体化设计	12
9.2	基于深度学习的多模态视觉感知与精确定位	13
9.3	路径规划与目标追踪控制	13
9.4	高效的任务开发与自动化验证流程	13

10 运行须知 **14**

10.1 启动方式 14

10.2 依赖项 14

1 项目简介

本项目聚焦于“无人机双机协同自主搜救”这一前沿挑战，设计并实现了一套面向复杂动态环境的异构无人机协同智能决策系统。系统由一架垂起固定翼无人机（VTOL）作为广域侦察单元和一架四旋翼无人机作为精准响应单元构成。

任务流程分为两个核心阶段：

- **阶段一：** VTOL 无人机利用其航程远、效率高的优势，在大范围内执行自主航路规划、动态模式切换、躲避禁飞区及多目标实时感知任务，通过搭载的 YOLOv8 视觉感知模块，实现对三类模拟人员的精确识别与世界坐标解算。
- **阶段二：** 在完成侦察并回传目标数据后，四旋翼无人机接替任务，基于接收到的目标信息执行高精度路径规划与末端视觉伺服控制，分别对静态及动态目标进行稳定追踪与抵近，最终在要求的高度上，完成关键信息的精确发布，全过程无需人工干预。

2 项目背景

无人机技术在环境监测、应急通信和抢险救灾等领域的应用正迅速深化。尤其在灾后搜救等复杂场景中，地面人员往往面临视野受限、地形阻碍及二次坍塌等重大安全风险。无人机凭借其空中优势，能够快速获取全局态势，显著提升搜救效率并降低人员风险。

然而，单一无人机平台难以兼顾大范围搜索效率与近距离精准操作的需求。为此，**异构双机协同**已成为解决此类复杂任务的前沿范式。该模式利用：

- VTOL 无人机兼具多旋翼垂直起降灵活性与固定翼长航时巡航能力的特点，执行广域快速扫描
- 四旋翼无人机优越的悬停与机动性能，执行抵近侦察与精准任务操作

本项目正是在此背景下，针对包含禁飞区、多类别动态/静态目标识别以及双机任务交接的复杂协同搜救场景，研发一套高效、鲁棒的智能决策与控制算法，旨在验证并提升异构无人机集群在该类任务中的自主协同作业能力。

3 项目目标

本项目的核心目标是设计、实现并验证一套应用于异构双机协同搜救任务的**智能决策与控制算法**。该系统旨在模拟真实搜救场景，验证无人机在 GPS 信号存在噪声、目标部分动态的挑战下，从起飞到任务完成的全流程自主作业能力。

具体子目标分解如下：

1. **高层任务规划与导航：**实现 VTOL 无人机的全自主起降、飞行模式无缝切换，以及融合了禁飞区的全局路径规划与航点跟踪能力。

2. **实时环境感知与目标定位**：开发基于 YOLOv8 的机载视觉感知模块，实现对多类别目标的实时检测、分类，并构建从像素坐标到全局地理坐标的精确解算模型。
3. **动态目标追踪与运动预测**：针对动态目标，设计一套轨迹数据采样与运动模型拟合算法，实现对目标未来位置的有效预测，为精准拦截提供决策依据。
4. **末端精准伺服与信息发布**：为四旋翼无人机开发基于视觉反馈的闭环伺服控制算法，实现对目标的稳定抵近、对心悬停，并在满足竞赛规则的高度与精度要求下，完成关键信息的发布。

4 项目实施

4.1 阶段一：任务启动与广域巡航

无人机 A (standard_vtol_0) 执行任务初始化序列：

1. 以多旋翼模式垂直起飞至预定安全高度
2. 执行多旋翼至固定翼的飞行模式转换
3. 激活全局路径规划模块，结合禁飞区约束，生成绕开禁飞区的无碰撞航路
4. 飞抵指定的 GPS 引导点

4.2 阶段二：目标搜索、识别与信息共享

抵达 GPS 引导点后：

- 无人机 A 进入搜索模式，执行预设的覆盖式路径规划航线确保对指定搜索区域的完全覆盖
- 激活机载视觉感知模块（基于 YOLOv8），对视频流实时处理，检测分类三类目标（危重、轻伤、健康）
- 通过坐标解算算法获取目标全局 Pose，并通过 ROS 话题发布（实现与无人机 B 的机间数据链模拟）
- 完成目标搜索与信息发布后，自主返航至旋翼模式区
- 执行固定翼至多旋翼模式转换，垂直降落并自动锁桨

4.3 阶段三：精准抵近与末端作业

无人机 A 安全着陆后，无人机 B (iris_0) 自动解锁起飞：

- 任务规划模块根据接收目标信息决策，规划前往健康人员（动态目标）的航路
- 抵达目标区域后，激活末端视觉引导系统：
 - 健康人员：进入目标状态估计阶段，通过连续视觉检测采样拟合运动轨迹，建立运动模型，计算最优拦截点并前出至该点上空，下降至 0.7 米高度等待目标进入指定区域后发布平台坐标
 - 重伤人员：激活视觉伺服控制逻辑，采用”对准-下降”递进策略，通过闭环控制修正水平位置，确保精确位于目标正上方，逐步下降至 0.5 米高度发布坐标

4.4 阶段四：任务完成与返航

无人机 B 完成对两个指定目标的任务后：

1. 执行返航程序，自主飞行返回旋翼模式区
2. 执行自动降落、锁桨
3. 标志着整个协同搜救任务的圆满完成

5 项目结构

5.1 项目文件结构

项目的所有文件位于工作区 catkin_ws/src/offboard_run 下：

```
1 .
2 |— CMakeLists.txt
3 |— README.md
4 |— models
5 |   |— red.pt
6 |   |— stage1.pt
7 |   |— stage2.pt
8 |   └— white.pt
9 |— msg
10 |— package.xml
11 |— score_check.py
12 |— scripts
13 |   |— __pycache__
14 |   └— detect_red.py
```

```
15 | |   |-- detect_stage1.py
16 | |   |-- detect_white.py
17 | |   |-- iris_commander.py
18 | |   |-- launch.sh
19 | |   |-- vtol_commander.py
20 | |-- src
```

5.2 部分介绍

src/offboard_run 目录包含主要的 ROS 包和代码文件：

- scripts 目录包含 Python 脚本：
 - vtol_commander.py: 固定翼无人机 A 阶段一和阶段二的起飞、搜索巡航、返回并降落的控制
 - detect_stage1.py: 固定翼无人机 A 阶段二的视觉识别，使用 YOLOv8 进行目标识别与坐标转换
 - iris_commander.py: 四旋翼无人机 B 阶段三和阶段四的起飞，接近目标、返回并降落的控制
 - detect_white.py: 四旋翼无人机 B 阶段三的健康人员视觉识别和精确降落控制
 - detect_red.py: 四旋翼无人机 B 阶段三的重伤人员视觉识别和精确降落控制
- CMakeLists.txt: 配置项目的 CMake 构建系统
- package.xml: 描述 ROS 包的元数据文件
- models: 包含 YOLOv8 模型文件，用于目标检测
- launch.sh: 一键启动整个项目的脚本
- score_check.py: 模拟评分的脚本

6 开发过程

6.1 整体流程

开发过程分为以下步骤：

1. 配置环境，学习 ROS 语法知识
2. 初步尝试无人机飞行，实现基本控制功能
3. 模块划分：

- 固定翼无人机：导航控制与视觉识别
- 四旋翼无人机：导航控制与精确降落

4. 完成两个无人机四个阶段的话题衔接

5. 整合测试

6.2 项目分工

按照任务类型划分：

- 无人机飞行控制组 (2 人):
 - 固定翼无人机起飞，飞行控制，避障与搜索路径规划
 - 四旋翼无人机起飞，飞行控制，避障路径规划，接近目标
- 视觉识别组 (2 人):
 - 视觉识别，坐标转换
 - 视觉模型的数据采集与训练
 - 根据视觉识别结果实现精确降落

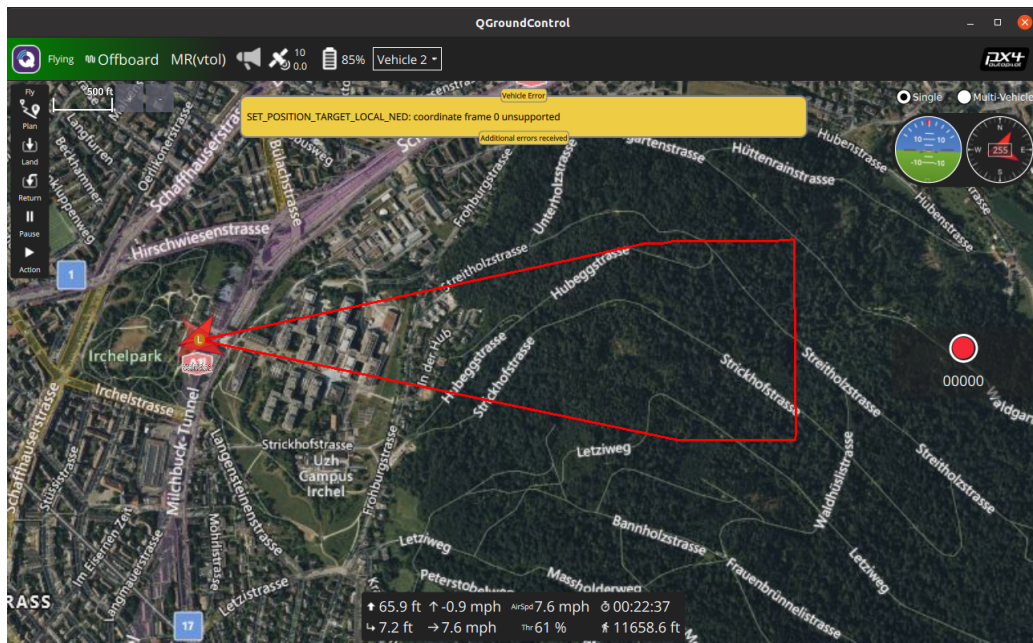
7 文件说明

7.1 vtol_commander.py（阶段一）

用于固定翼无人机的导航控制，实现：

- 根据 GPS 坐标点绕过禁飞区到达目标区域
- 激活第一阶段的视觉识别进程
- 目标搜索

7.1.1 整体路线预览



7.1.2 起飞降落逻辑

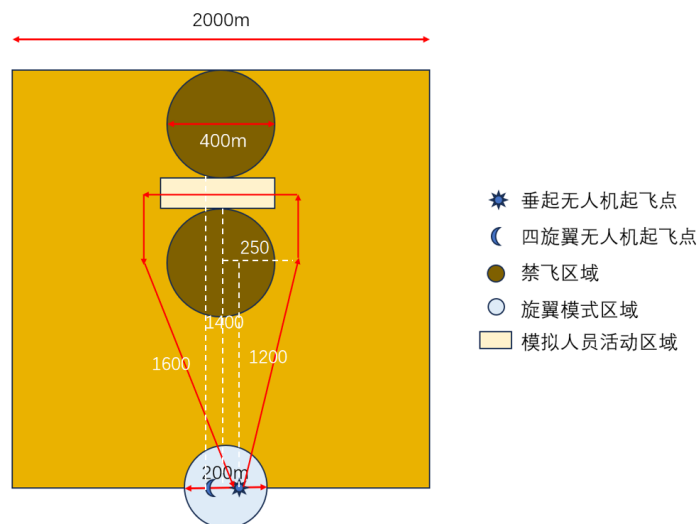
- 多旋翼模式起飞到 20 米高度
- 切换为固定翼模式执行任务
- 返回旋翼模式区切换回多旋翼模式并降落锁桨

```
jane@ubuntu: ~/catkin_ws/src/offboard_run/scripts
[终端] VTOL 控制脚本 (vtol_commander.py) 启动中...
[INFO] [1755148453.312362, 10255.008000]: Commander node initialized.
[INFO] [1755148453.315318, 10255.010000]: Waiting for first position message...
[INFO] [1755148453.378914, 10255.060000]: Position received.
[INFO] [1755148453.380373, 10255.061000]: Waiting for FCU connection...
[INFO] [1755148453.381767, 10255.062000]: FCU connected.
[INFO] [1755148453.382638, 10255.063000]: Attempting to activate OFFBOARD mode and arm...
[INFO] [1755148469.059576, 10260.115000]: OFFBOARD mode enabled
[INFO] [1755148488.763528, 10265.177000]: Vehicle armed
[INFO] [1755148489.963801, 10265.513000]: Vehicle is in OFFBOARD mode and armed.
[INFO] [1755148489.965705, 10265.513000]: Takeoff to height: 20.00
[INFO] [1755148529.651271, 10276.514000]: Takeoff complete. Hovering at 19.51 meters.
[INFO] [1755148541.368883, 10279.514000]: Switching to FIXED-WING mode...
[INFO] [1755148543.039661, 10280.015000]: Rotor mode zone center set at (-0.05, -0.04)
[INFO] [1755148543.042176, 10280.016000]: switching to fixed wing mode...
[INFO] [1755148545.634611, 10281.022000]: --- Starting Cruise Mission ---
[INFO] [1755148545.647766, 10281.022000]: change to altitude: 40.00
```

```
jane@ubuntu: ~/catkin_ws/src/offboard_run/scripts
[INFO] [1755149439.857883, 10534.905000]: Step 1: Returning to rotor zone center in fixed-wing mode.
[INFO] [1755149439.860758, 10534.905000]: Moving to the target position (-0.05, -0.04, 20.01)
[INFO] [1755149844.431043, 10642.755000]: Move command finished for target (-3.16, -0.55, 20.68).
[INFO] [1755149844.435367, 10642.760000]: Step 2: Arrived at zone center. Switching to multicopter mode.
[INFO] [1755149844.443886, 10642.760000]: Switching to MULTIROTOR mode...
[INFO] [1755149864.432760, 10648.262000]: Step 3: Commanding automatic land.
[INFO] [1755149864.434615, 10648.263000]: Commanding LAND
[INFO] [1755149864.453910, 10648.273000]: Land command sent successfully.
[INFO] [1755149864.460965, 10648.275000]: Waiting for landing and disarm...
[INFO] [1755149945.984092, 10671.282000]: Landed and disarmed successfully. Mission complete.
[INFO] [1755149945.988326, 10671.283000]: Commander node terminated.
GAZEBO_PLUGIN_PATH :/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo:/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo
GAZEBO_MODEL_PATH :/home/jane/PX4_Firmware//Tools/sitl_gazebo/models:/home/jane/PX4_Firmware//Tools/sitl_gazebo/models
LD_LIBRARY_PATH /home/jane/catkin_ws/devel/lib:/opt/ros/noetic/lib:/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo:/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo
```

7.1.3 避障策略

- 保持安全距离避开禁飞区
- 设置较高飞行速度避免碰撞



7.2 detect_stage1.py (阶段二)

实现利用 vtol 无人机遥感数据进行目标识别定位:

- 数据读取: 使用消息队列保存同一时刻的图像和位置信息
- 识别功能: 基于 YOLOv8 模型实时检测和分类目标
- 坐标计算: 基于遥感坐标转换模型计算物体具体坐标
- 数据处理: 静止目标使用均值滤波, 运动目标选择靠近图像中心的坐标

视觉识别模型:

1. 采集飞行遥感数据：

利用 `save.py` 脚本对遥感数据进行采集

2. 标注遥感数据：

利用 Roboflow 平台 进行数据标注

3. 对基础模型进行再训练：

4. 利用训练好的模型：

详情见于源码

遥感坐标转换模型：

1. 像素坐标转相机方向向量：

$$\mathbf{p}_{\text{cam}} = K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}$$

2. 相机坐标系到机体坐标系：

$$\mathbf{p}_{\text{body}} = R_{\text{cam} \rightarrow \text{body}} \mathbf{p}_{\text{cam}} + \mathbf{t}_{\text{cam} \rightarrow \text{body}}$$

3. 机体坐标系到世界坐标系：

$$\mathbf{p}_{\text{target}} = \mathbf{p}_{\text{UAV}} + R_{\text{body} \rightarrow \text{world}} \mathbf{p}_{\text{body}}$$

4. 三维坐标计算（射线与地面交点）：

$$t = \frac{\text{ground}_z - \text{cam}_z}{\text{ray}_{\text{world}}[2]}, \quad P = \text{Cam}_{\text{position}} + t \cdot \text{ray}_{\text{world}}$$

5. 误差修正：

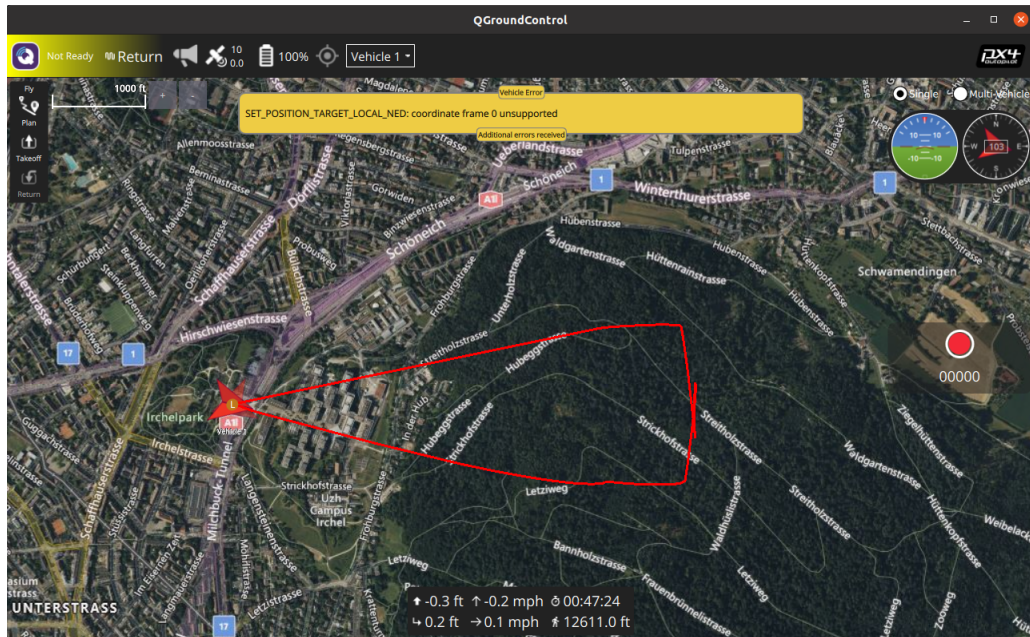
$$P = P + v_{\text{error}}$$

7.3 iris_commander.py（阶段三）

用于四旋翼无人机的导航控制：

- 根据阶段二识别的坐标绕过禁飞区
- 激活第四阶段的视觉识别
- 精确定位与降落

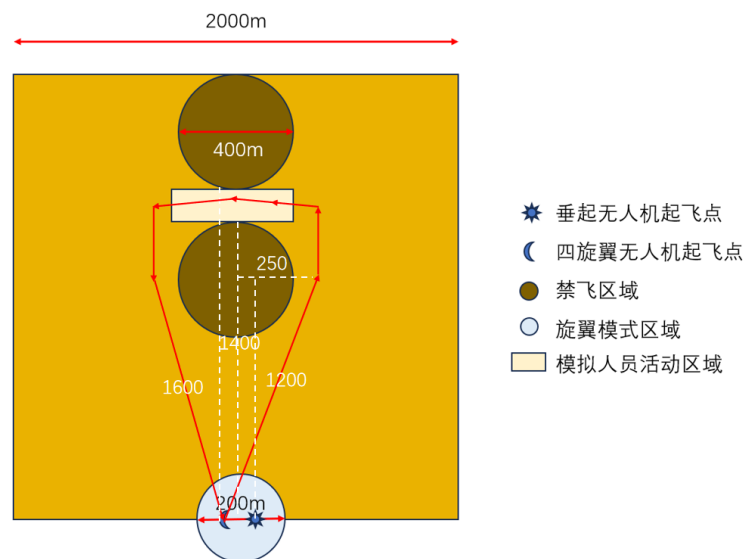
7.3.1 整体路线预览（示例）



7.3.2 控制逻辑

通过 ROS 话题直接控制飞行、悬停和移动

7.3.3 避障策略



7.4 detect_white.py & detect_red.py（阶段四）

无人机到达目标区域后的递进侦查：

- 目标识别线程：
 - 保存遥感图像和飞机位姿信息

- 使用 YOLOv8 模型进行目标标注
- 通过遥感视线模型计算目标位置
- 误差修正和数据同步
- UAV 控制线程：
 - 健康人员（动态目标）：
 - * SEARCHING: 移动到目标位置并下降
 - * MODELING: 采样并拟合运动轨迹
 - * INTERCEPTING: 移动到拦截点上方
 - * DESCENDING_TO_WAIT: 下降到 0.7 米
 - * WAITING_FOR_TARGET: 等待目标通过并发布坐标
 - 重伤人员（静态目标）：
 - * SEARCHING: 微调至目标正上方
 - * ALIGNING: 下降一段距离
 - * 重复直到下降到 0.5 米高度发布坐标

8 关键代码及注释

详见各脚本文件源代码（因篇幅限制，此处省略具体代码）

9 创新点

9.1 异构双机协同任务规划与 GNC 一体化设计

- 构建 VTOL 与四旋翼高效协同架构
- 设计任务分配与信息流交接机制
- 实现任务规划与 GNC 系统一体化设计
 - 状态机负责顶层任务决策
 - 通过 MAVROS 接口转化为 PX4 飞控指令
- 融入禁飞区规避策略确保飞行安全

9.2 基于深度学习的多模态视觉感知与精确定位

- 集成 YOLOv8 深度学习模型
- 自定义数据集微调训练（详见Roboflow 平台）
- 构建鲁棒坐标解算流程：
 - 基于相机针孔模型反投影
 - 使用 `message_filters` 进行时空同步

9.3 路径规划与目标追踪控制

针对不同类型目标设计差异化策略：

- 静态目标（危重人员）：
 - 基于位置的视觉伺服控制
 - 高频视觉反馈循环修正水平误差
 - ”对准-下降”复合式降落
- 动态目标（健康人员）：
 - ”在线建模-预测-拦截”智能决策
 - 轨迹数据采集与最小二乘拟合
 - 运动模型预测最佳拦截点

9.4 高效的任务开发与自动化验证流程

- `launch.sh` 脚本实现：
 - Gazebo 仿真环境一键部署
 - PX4 飞控与所有节点自动启动
- `score_check.py` 自动化评分：
 - 订阅关键 ROS 话题
 - 与 Gazebo 真值比对
 - 量化评估任务执行效果

10 运行须知

10.1 启动方式

```
1 # 启动地面站
2 cd ~/Downloads # 地面站安装位置
3 ./QGroundControl.AppImage
4
5 # 激活启动脚本
6 cd ~/catkin_ws/src/offboard_run/scripts
7 ./launch.sh
```

详细问题参考使用说明.pdf

10.2 依赖项

- OpenCV (建议 4.10.0):

```
1 sudo apt-get install python3-opencv
```

- YOLOv8 (ultralytics 包):

```
1 pip install ultralytics
```

- cv_bridge:

```
1 sudo apt-get install ros-<distro>-cv-bridge
```

- gazebo_msgs:

```
1 sudo apt-get install ros-<distro>-gazebo-msgs
```